CMPE322 OPERATING SYSTEMS PROJECT

Shooter

Fifo & Signal Input

Mehmet Saraçoğlu

Designed as course material for use in the CMPE322 Operating Systems lectures.

Deadline: **25.10.2026, 23:59 (GMT+3)**

Contents

1 Introduction **2**

2 Setup **2**

2.1 Debian 13 and installation . . . 2

2.2 Required packages . . . 2

2.3 First run . . . 2

3 Project Description **3**

3.1 The game . . . 3

3.2 Frame rate and polling . . . 3

3.3 The input interface (`input.h`) . . . 4

3.4 CLI input mechanism . . . 4

3.5 Dynamic loading . . . 5

3.6 Functions to implement . . . 5

3.7 Goal: the control script . . . 6

3.8 Building and running . . . 7

3.9 Project tree . . . 8

4 Submission Rules **9**

Acknowledgments **9**

---

1. Introduction

In this project, you will implement an input mechanism for a fully implemented 2D space shooter game. The goal is to reach the exit station.

A complete input mechanism is already implemented in `input_keyboard.c`, which allows you to play the game with regular keyboard input. You can use it to test and explore the in-game mechanics.

In this project, you will implement a text-based input mechanism to win the game. At the highest level, a script will command your ship. You will write both the script carrying the winning moves and the intermediary mechanisms.

2. Setup

2.1 Debian 13 and installation

In this project, we will use Debian 13 as the underlying operating system. Install it on your computer or in a virtual machine (VM), and test your code there. Download it from these links:

CPU architecture	Netinst image
amd64	https://cdimage.debian.org/debian-cd/current/amd64/iso-cd/debian-13.7.0-amd64-netinst.iso
arm64	https://cdimage.debian.org/debian-cd/current/arm64/iso-cd/debian-13.7.0-arm64-netinst.iso

The tests will be run on Debian 13 instances, so make sure your solution works correctly on this specific distribution.

2.2 Required packages

After installing Debian, install these packages with the `apt install` command:

- `build-essential`
- `libncurses-dev`

Before installing anything, make sure you update the apt package cache with `apt update`.

2.3 First run

Copy your game files into the Debian instance you are using, then run these commands:

```
cd 2D_SPACE_SHOOTER
make keyboard
./run.sh <your_student_ID> keyboard
```

Keys: `W A S D` move, `L` shoot, `J` shield, `K` gather, `P` pause, `Q` quit. Use a large terminal window.

`input_keyboard.c` is provided for practical purposes: play the level yourself to learn the game and find a route to the exit, and read the code as a complete, working example of an input source. Your source fills the same interface.

3. Project Description

3.1 The game

These are the existing entities in the game:

Entity	Meaning
Your ship	Starts next to the `START` station. Has a shield; each hit costs shield energy, and at 0 you are shot down.
`END` station	The exit. **You win by standing next to it.**
`RECHARGE` station	Standing next to it, *gather* moves energy from it into your shield.
`ENEMY_SPAWN` station	A nest: releases enemies while it has energy.
Enemy ship	Hunts you, and fires when lined up with you.
Planet	Blocks ships and absorbs bullets.

These are the available operations:

Op	Effect
`OP_UP`, `OP_DOWN`, `OP_LEFT`, `OP_RIGHT`	Turn and move in that direction. A direction that keeps coming keeps the ship moving.
`OP_SHOOT`	Fire a bullet in the direction the ship faces.
`OP_SHIELD_ACTIVATE`	Arm a block that absorbs the next hit.
`OP_GATHER`	Take energy from an adjacent `RECHARGE` station.
`OP_PAUSE`	Toggle pause / continue.
`OP_QUIT`	End the game.

The level file is `levels/level01.txt`.

> You have your own scenario: each student will have a different solution. You have to find **YOUR SOLUTION**, so make sure you are running the game with your own student ID.

3.2 Frame rate and polling

The game runs at **60 frames per second**. On every frame the game asks the input source for ops with `input_poll`, giving it a time limit, `timeout_ms`: the time left until the next frame must be drawn. The polling is a basic sleeping mechanism triggered by input: while nothing has arrived, the source sleeps; it returns **as soon as an op arrives** or **when `timeout_ms` has passed**, whichever comes first. The first rule makes ops take effect without delay; the second keeps the game drawing its frames on time, since the game cannot continue until `input_poll` returns.

3.3 The input interface (`input.h`)

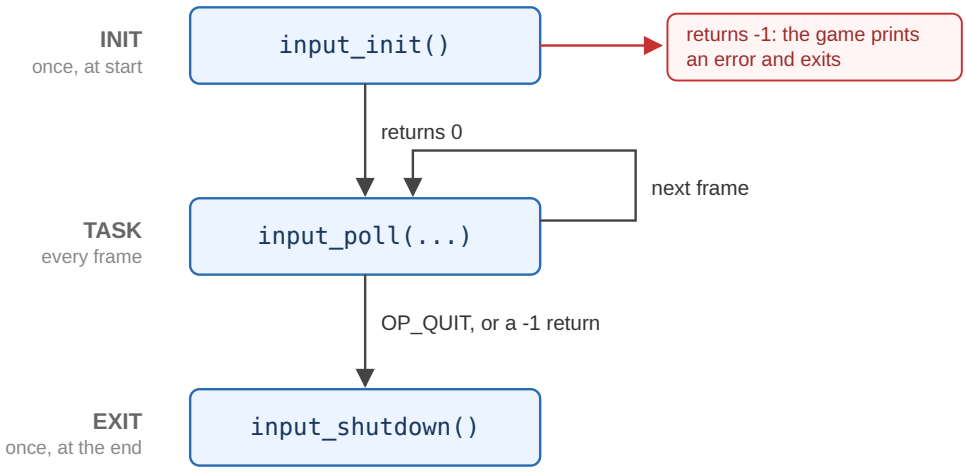
The game communicates with an input source only through three functions.

```c
// Initialize.
int input_init(void);

// Clean up.
void input_shutdown(void);

// Hand out ops.
int input_poll(int timeout_ms, enum input_op *ops, int max_ops);
```

`input_init` initializes the input mechanism, `input_poll` provides the processed data to the game's main loop, and `input_shutdown` cleans up and exits cleanly.



3.4 CLI input mechanism

The main communication between the shell script you will write and the running program instance will be over a fifo (see the *named pipe* concept) and signals.

Signals, sent to the running `game` process:

Signal	Op
`SIGUSR1` (`SIG_PAUSE`)	`OP_PAUSE`
`SIGTERM` (`SIG_QUIT`)	`OP_QUIT`

The fifo, one character per op:

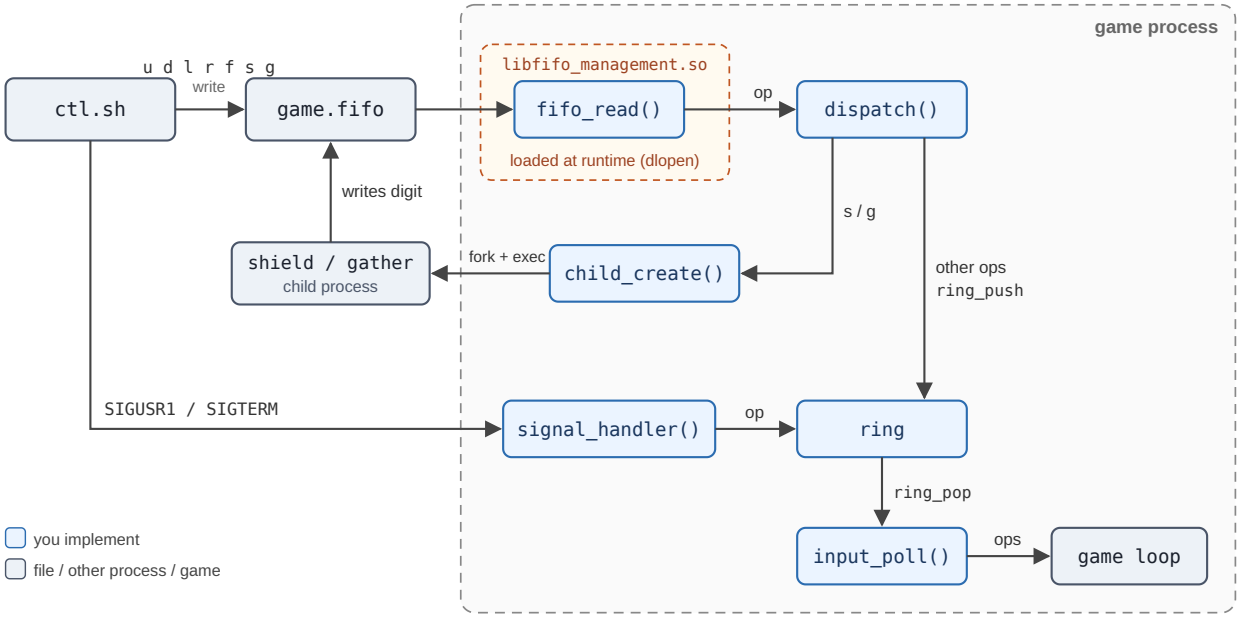
Char	Op	Char	Op
`u`	up	`f`	shoot (*fire*)
`d`	down	`s`	shield
`l`	left	`g`	gather
`r`	right		

The fifo, `implementation/game.fifo`, is created by your code inside the `implementation/` folder. One character per op; any other character is ignored. `s` and `g` are **spawn requests**: for each one

the game starts a child process (`shield` or `gather`). The child writes its op's enum value as a digit (`FIFO_OP_CHAR(op)`) back into the fifo, sleeps for its duration (shield 1 s, gather 0 s) and exits; only that digit becomes the op. The game never waits for these children; they run in parallel.

The fifo is managed through a **shared library**. The header of that shared library is given to you in `fifo_management.h`. It is compiled under the name `libfifo_management.so`.

Ops gathered from both paths are queued in a **circular buffer** (ring), and `input_poll` hands them out. The produced output is read by the main loop of the game.



A signal may arrive while fifo input is being processed and override a fifo entry (a race condition). The game's split of time into 1/60 s ticks handles this naturally, so you are **NOT** expected to write mutual exclusion mechanisms for it.

3.5 Dynamic loading

The shared library (`libfifo_management.so`) you implement will be used dynamically: it is **loaded at runtime**. Linking that library will cost you points.

Forbidden	Dynamic linking
Expected	Dynamic loading

3.6 Functions to implement

The headers `fifo_management.h` and `input_student.h` are given and **must not be changed**. Their comments are part of the specification.

fifo_management.c → libfifo_management.so

Function	Description
int fifo_init(void)	Creates and opens the fifo; the game must never block on it. Returns 0, or −1.
int fifo_read(void)	Next op in the fifo, FIFO_SPAWN(op) for a spawn request, or −1 if empty. Never blocks.
int fifo_destroy(void)	Closes and removes the fifo. Returns 0.

shield.c → shield
gather.c → gather

Each writes FIFO_OP_CHAR(op) of its op into the fifo, sleeps for its duration, and exits.

input_student.c → linked into game (done by the Makefile)

Function	Description
ring_init, ring_push, ring_pop	Empty the ring; queue an op (dropped if full); take the oldest op (1, or 0 if empty).
signal_handler	Turns SIG_PAUSE / SIG_QUIT into its op in the ring.
signal_register, signal_unregister	Install the handlers; restore the defaults.
so_load, so_unload, so_call	Load the library and check its symbols; unload it; call one of its functions by name.
child_create	Start the shield / gather process with fork + exec; never wait for it.
dispatch	Read everything waiting in the fifo: spawn requests start a child, other ops go into the ring.
input_init, input_shutdown, input_poll	The input.h interface on top of all of the above.

3.7 Goal: the control script

Write implementation/ctl.sh, a bash script that plays **level 1** and **wins**: the game must reach the YOU WIN screen.

- ./run.sh <student_id> builds the game, starts your script **in the background**, then starts the game with your ID. Your script may start before the fifo exists; it must handle this case.
- Your pattern must win **with your own student ID** (see Section 3.1).
- Movement, shooting, shield and gather: characters written into implementation/game.fifo.
- Pause and quit: signals sent to the game process.
- Timing matters. Find your route in keyboard mode first.

No exiting. Once the game is decided, it stays on the **YOU WIN** or **YOU LOSE** screen; your procedure must leave it there. Do **not** quit or kill the game at any step: not in the middle of the run, not after the outcome, not from your script and not from your input source. The game is closed by the evaluator. **Exiting arbitrarily is penalized.**

3.8 Building and running

```
make                    # your source: game + libfifo_management.so + shield + gather, in one step
make keyboard           # keyboard source
make clean              # removes everything built

./run.sh 2017400021               # your source + implementation/ctl.sh on level 1
./run.sh 2017400021 keyboard      # you play your scenario
./run.sh 2017400021 fifo 1 myctl  # your source + implementation/myctl.sh on level 1
```

In all examples, `2017400021` stands for **your own student ID**. Always start the game through **`./run.sh`**.

While the game runs, you can also drive it by hand from a second terminal in the same directory: `printf u > implementation/game.fifo`, `kill -USR1 $(pgrep -x game)`.

Command-line arguments

All arguments are **positional**: their meaning is given by their position, not by a name. An argument can be given only if every argument before it is also given; an omitted argument takes its default.

`./run.sh <student_id> [mode] [level] [control]` — builds the game and starts it in the current terminal. `<...>` is required, `[...]` is optional.

Pos.	Argument	Values	Default	Meaning
1	`student_id`	exactly 10 digits	**required**	Your student ID; it selects your scenario.
2	`mode`	`fifo` or `keyboard`	`fifo`	`fifo`: your input source, driven by the control script. `keyboard`: you play.
3	`level`	level number: `1` or `01`	`1`	Plays `levels/level<NN>.txt`, e.g. `1` → `levels/level01.txt`.
4	`control`	script name, without `.sh`	`ctl`	Runs `implementation/<control>.sh` in the background. Used only in `fifo` mode; ignored in `keyboard` mode.

Command	Mode	Level file	Control script
`./run.sh 2017400021`	`fifo`	`levels/level01.txt`	`implementation/ctl.sh`
`./run.sh 2017400021 fifo 1 myctl`	`fifo`	`levels/level01.txt`	`implementation/myctl.sh`
`./run.sh 2017400021 keyboard`	`keyboard`	`levels/level01.txt`	none
`./run.sh`	refused: usage message, the game does not start		

3.9 Project tree

What you receive:

```
2D_SPACE_SHOOTER/
├── Makefile               make / make keyboard / make clean
├── run.sh                 build + run with a level and a control script
├── main.c                 main loop
├── input.h                input interface
├── input_keyboard.c       keyboard source
├── control.[ch]  realtime.[ch]  space.[ch]  entity.[ch]  ship.[ch]  bullet.[ch]
├── planet.[ch]  station.[ch]  level.[ch]  renderer.[ch]  window.[ch]  hud.[ch]
├── levels/
│   └── level01.txt
└── implementation/
    ├── fifo_management.h  given; must not be changed
    ├── input_student.h    given; must not be changed
    ├── fifo_management.c  empty; you fill it
    ├── input_student.c    empty; you fill it
    ├── shield.c           empty; you fill it
    ├── gather.c           empty; you fill it
    └── ctl.sh             empty; you fill it
```

`implementation/` is the only folder you work in. It contains the **five mutable files** marked *empty* above; fill them in place. Do not rename any file, and do not change the content of any other file. **Every other file is immutable, including the `Makefile`.** Do not touch the other files even if it seems that something might work that way: your files are evaluated with the original package.

Every file created by your work must be inside `implementation/`. This covers both compilation outputs and runtime files:

- compilation outputs: `libfifo_management.so`, `shield`, `gather` and the object files of `input_student.c` (the given `Makefile` already places them there);
- runtime files: the fifo, `implementation/game.fifo` (its path is `FIFO_PATH` in `fifo_management.h`), and **any other file** your code or your script creates, such as logs or temporary files.

Your code and your script must **not create, modify or delete any file outside `implementation/`**. The core game keeps its own outputs (`game` and `build/`) in the package root; these are produced by the given `Makefile`, not by you. The evaluation environment keeps its own files outside `implementation/`; any file your work creates or touches there is **penalized**.

Until the five files are filled, `make` stops with errors; `make keyboard` always works.

4. Submission Rules

- Submit **one `.zip` file** named `<student_id>.zip`, containing the `implementation/` folder completely.
- **No binaries and no compiled files.** Run `make clean` before packing; it removes `game` and `build/` from the package root, and `libfifo_management.so`, `shield`, `gather`, the `input_student` object files and `game.fifo` from `implementation/`.
- Any requirement mentioned in this document will be tested and evaluated. Your `implementation/` folder is copied into a fresh copy of the `2D_SPACE_SHOOTER` directory on Debian 13, and the following commands are run, with `<student_id>` taken from the name of your `.zip` file:

```
make clean && make      # must build with no errors
./run.sh <student_id>   # must reach the YOU WIN screen and stay there
```

  Warnings under `-Wall -Wextra` cost points. A game that exits by itself at any step (quit, kill, crash) instead of staying on the `YOU WIN` / `YOU LOSE` screen is penalized.
- The use of AI tools is not limited at any point while implementing this project. However, using them directly without any knowledge will most likely result in spaghetti code and make you lose an enormous amount of time. First research the topics you need, understand the concepts well, and then move forward.
- This is an **individual project**. No cheating or code sharing: code sharing will be checked, and if suspicious code is detected, you may have an oral exam about your project implementation. Plagiarism rules of the course apply.
- Deadline: **25.10.2026, 23:59 (GMT+3)**, i.e. 25 October 2026, 23:59 Istanbul time.

Acknowledgments

The author thanks **Ahmet Emin Bilen** and **Ahmet Cem Çalık** for their assistance in the preparation of this project.